Database and Security

Minimal prototype specification | Markdown is the source of truth

Use Supabase PostgreSQL. Apply all schema changes through a migration that can be rerun in a fresh project. Enable Row Level Security on every application table before the app is shared.

Tables

profiles

- `id uuid primary key references auth.users(id) on delete cascade`
- `display_name text not null`
- `avatar_url text null`
- `created_at timestamptz not null default now()`

Create or upsert a profile for each authenticated user. A simple auth-user trigger is acceptable.

groups

- `id uuid primary key default gen_random_uuid()`
- `name text not null`
- `owner_id uuid not null references profiles(id)`
- `invite_code text not null unique`
- `created_at timestamptz not null default now()`

group_members

- `group_id uuid not null references groups(id) on delete cascade`
- `user_id uuid not null references profiles(id) on delete cascade`
- `joined_at timestamptz not null default now()`
- Primary key: `(group_id, user_id)`

The owner must also have a membership row.

problems

- `id uuid primary key default gen_random_uuid()`
- `group_id uuid not null references groups(id) on delete cascade`
- `title text not null`
- `url text not null`
- `platform text not null default 'LeetCode'`
- `difficulty text not null check (difficulty in ('Easy', 'Medium', 'Hard'))`
- `problem_date date not null`
- `note text null`
- `created_by uuid not null references profiles(id)`
- `created_at timestamptz not null default now()`
- `updated_at timestamptz not null default now()`

- Unique constraint: `(group_id, problem_date)`

`completions`

- `id uuid primary key default gen_random_uuid()`
- `problem_id uuid not null references problems(id) on delete cascade`
- `user_id uuid not null references profiles(id) on delete cascade`
- `completed_at timestamptz not null default now()`
- Unique constraint: `(problem_id, user_id)`

`comments`

- `id uuid primary key default gen_random_uuid()`
- `problem_id uuid not null references problems(id) on delete cascade`
- `user_id uuid not null references profiles(id) on delete cascade`
- `message text not null check (char_length(trim(message)) between 1 and 1000)`
- `created_at timestamptz not null default now()`

Useful indexes

- `group_members(user_id)`
- `problems(group_id, problem_date desc)`; the unique constraint may already provide this index.
- `completions(problem_id)`
- `completions(user_id)`
- `comments(problem_id, created_at)`

Do not add derived heatmap or daily-summary tables.

Required Row Level Security behavior

Policies may use small `security definer` membership helper functions if needed to avoid policy recursion. Set a safe `search_path` on such functions.

Profiles

- Authenticated users may read profiles only when needed to display members of a group they share.
- A user may update only their own profile.

Groups

- A member may read a group they belong to.
- An authenticated user may create a group with themselves as owner.
- Only the owner may update or delete the group.

Group members

- Members may read membership rows for their own groups.
- A user may insert only a membership for themselves and only when the supplied invite code identifies the group. Prefer a small database function/RPC for atomic invite-code joining if direct insert cannot be secured cleanly.
- A user may remove only their own membership. Prevent or handle the owner leaving in V1.

- Only the owner may remove another member.

Problems

- Group members may read problems in their group.
- Only the group owner may insert, update, or delete problems for that group.
- `created_by` must equal the authenticated user on insert.

Completions

- Group members may read completions for problems in their group.
- A user may insert or delete only their own completion and only for a problem in a group they belong to.
- No user may update a completion to another user/problem.

Comments

- Group members may read comments for problems in their group.
- A user may insert only their own comment inside a group they belong to.
- Deleting or editing comments is not required. If deletion is added, allow only the author or group owner.

Integrity checks

- Attempting a duplicate group membership fails harmlessly.
- Attempting a duplicate completion fails harmlessly.
- A non-member cannot read a group through direct API calls, not just through hidden UI.
- A member cannot create a problem by calling Supabase directly.
- A user cannot create or delete another user's completion.
- Deleting a problem removes its completions and comments.
- Heatmap and daily counts are query results, never independent stored state.

Data handling

- Store timestamps in UTC and format them in the browser.
- Store daily assignment as `date`, not a timestamp.
- Generate invite codes with sufficient randomness; do not use sequential IDs.
- Never expose or use the Supabase service-role key in frontend code.